

# Analyse C1 et C2 de l'API bancaire

Suite du devoir de test logiciel

**Projet :** API de transaction bancaire

**Étudiant :** Demano Bramael

**Matricule :** 25G2033

**Code source analysé :** `comptes/views.py`

## 1 Fonctionnalités du logiciel

Les cinq fonctionnalités principales de l'API bancaire sont les suivantes :

1. **Créer un compte bancaire** : enregistre un nouveau compte et crée automatiquement un dépôt initial si le solde d'ouverture est positif.
2. **Lister les comptes bancaires** : retourne tous les comptes actifs disponibles dans le système.
3. **Consulter un compte bancaire** : affiche les informations détaillées d'un compte et l'historique associé.
4. **Effectuer un dépôt** : ajoute un montant au solde du compte et enregistre une transaction de type DEPOT.
5. **Effectuer un retrait** : retire un montant si le compte existe, est actif et possède un solde suffisant, puis enregistre une transaction de type RETRAIT.

## 2 Rappel sur C1 et C2

**C1** correspond à la **couverture des instructions** : chaque instruction du programme doit être exécutée au moins une fois.

**C2** correspond à la **couverture des décisions** : chaque décision doit être évaluée au moins une fois à vrai et au moins une fois à faux.

## 3 Graphes de flot de contrôle

Pour rester fidèle au code source, chaque graphe est donné sous forme de nœuds et d'arcs. Cela permet ensuite d'identifier clairement les chemins d'exécution.

### 3.1 Fonction 1 : `creer_compte`

#### Nœuds

- N1 : début de la fonction.
- N2 : création du serializer avec `request.data`.
- N3 : test `serializer.is_valid()`.
- N4 : sauvegarde du compte.
- N5 : sérialisation de la réponse.
- N6 : retour HTTP 201.
- N7 : retour HTTP 400.

**Arcs** :  $N1 \rightarrow N2$ ,  $N2 \rightarrow N3$ ,  $N3(\text{vrai}) \rightarrow N4$ ,  $N4 \rightarrow N5$ ,  $N5 \rightarrow N6$ ,  $N3(\text{faux}) \rightarrow N7$ .

#### Chemins

- P1 :  $N1 \rightarrow N2 \rightarrow N3(\text{vrai}) \rightarrow N4 \rightarrow N5 \rightarrow N6$
- P2 :  $N1 \rightarrow N2 \rightarrow N3(\text{faux}) \rightarrow N7$

### 3.2 Fonction 2 : liste\_comptes

#### Nœuds

- N1 : début.
- N2 : récupération des comptes actifs.
- N3 : sérialisation.
- N4 : retour HTTP 200.

**Arcs** :  $N1 \rightarrow N2$ ,  $N2 \rightarrow N3$ ,  $N3 \rightarrow N4$ .

#### Chemin unique

- P1 :  $N1 \rightarrow N2 \rightarrow N3 \rightarrow N4$

### 3.3 Fonction 3 : detail\_compte

#### Nœuds

- N1 : début.
- N2 : appel à `get_compte(pk)`.
- N3 : test `compte is None`.
- N4 : retour HTTP 404.
- N5 : sérialisation du compte.
- N6 : retour HTTP 200.

**Arcs** :  $N1 \rightarrow N2$ ,  $N2 \rightarrow N3$ ,  $N3(\text{vrai}) \rightarrow N4$ ,  $N3(\text{faux}) \rightarrow N5$ ,  $N5 \rightarrow N6$ .

#### Chemins

- P1 :  $N1 \rightarrow N2 \rightarrow N3(\text{vrai}) \rightarrow N4$
- P2 :  $N1 \rightarrow N2 \rightarrow N3(\text{faux}) \rightarrow N5 \rightarrow N6$

### 3.4 Fonction 4 : effectuer\_depot

#### Nœuds

- N1 : début.
- N2 : récupération du compte.
- N3 : test `compte is None`.
- N4 : retour HTTP 404.
- N5 : test `not compte.actif`.
- N6 : retour HTTP 400 compte inactif.
- N7 : création du serializer d'opération.
- N8 : test `serializer.is_valid()`.
- N9 : retour HTTP 400 données invalides.
- N10 : lecture du montant et de la description.
- N11 : bloc transactionnel et mise à jour du solde.
- N12 : création de la transaction `DEPOT`.
- N13 : retour HTTP 200.

**Arcs** :  $N1 \rightarrow N2$ ,  $N2 \rightarrow N3$ ,  $N3(\text{vrai}) \rightarrow N4$ ,  $N3(\text{faux}) \rightarrow N5$ ,  $N5(\text{vrai}) \rightarrow N6$ ,  $N5(\text{faux}) \rightarrow N7$ ,  $N7 \rightarrow N8$ ,  $N8(\text{faux}) \rightarrow N9$ ,  $N8(\text{vrai}) \rightarrow N10$ ,  $N10 \rightarrow N11$ ,  $N11 \rightarrow N12$ ,  $N12 \rightarrow N13$ .

#### Chemins

- P1 :  $N1 \rightarrow N2 \rightarrow N3(\text{vrai}) \rightarrow N4$
- P2 :  $N1 \rightarrow N2 \rightarrow N3(\text{faux}) \rightarrow N5(\text{vrai}) \rightarrow N6$
- P3 :  $N1 \rightarrow N2 \rightarrow N3(\text{faux}) \rightarrow N5(\text{faux}) \rightarrow N7 \rightarrow N8(\text{faux}) \rightarrow N9$
- P4 :  $N1 \rightarrow N2 \rightarrow N3(\text{faux}) \rightarrow N5(\text{faux}) \rightarrow N7 \rightarrow N8(\text{vrai}) \rightarrow N10 \rightarrow N11 \rightarrow N12 \rightarrow N13$

### 3.5 Fonction 5 : effectuer\_retrait

#### Nœuds

- N1 : début.
- N2 : récupération du compte.
- N3 : test `compte is None`.
- N4 : retour HTTP 404.
- N5 : test `not compte.aktif`.
- N6 : retour HTTP 400 compte inaktif.
- N7 : création du serializer d'opération.
- N8 : test `serializer.is_valid()`.
- N9 : retour HTTP 400 données invalides.
- N10 : lecture du montant et de la description.
- N11 : test `compte.solde < montant`.
- N12 : retour HTTP 400 solde insuffisant.
- N13 : bloc transactionnel et décréement du solde.
- N14 : création de la transaction `RETRAIT`.
- N15 : retour HTTP 200.

**Arcs :** N1→N2, N2→N3, N3(vrai)→N4, N3(faux)→N5, N5(vrai)→N6, N5(faux)→N7, N7→N8, N8(faux)→N9, N8(vrai)→N10, N10→N11, N11(vrai)→N12, N11(faux)→N13, N13→N14, N14→N15.

#### Chemins

- P1 : N1 → N2 → N3(vrai) → N4
- P2 : N1 → N2 → N3(faux) → N5(vrai) → N6
- P3 : N1 → N2 → N3(faux) → N5(faux) → N7 → N8(faux) → N9
- P4 : N1 → N2 → N3(faux) → N5(faux) → N7 → N8(vrai) → N10 → N11(vrai) → N12
- P5 : N1 → N2 → N3(faux) → N5(faux) → N7 → N8(vrai) → N10 → N11(faux) → N13 → N14 → N15

## 4 Jeux de tests dérivés des chemins

| Cas | Fonction            | But du test                  | Chemin visé                        | Critère |
|-----|---------------------|------------------------------|------------------------------------|---------|
| T1  | Créer un compte     | données valides              | P1 de <code>creer_compte</code>    | C1, C2  |
| T2  | Créer un compte     | données invalides ou doublon | P2 de <code>creer_compte</code>    | C2      |
| T3  | Lister les comptes  | appel normal de la liste     | P1 de <code>liste_comptes</code>   | C1      |
| T4  | Consulter un compte | identifiant existant         | P2 de <code>detail_compte</code>   | C1, C2  |
| T5  | Consulter un compte | identifiant inexistant       | P1 de <code>detail_compte</code>   | C2      |
| T6  | Dépôt               | compte introuvable           | P1 de <code>effectuer_depot</code> | C2      |
| T7  | Dépôt               | compte inaktif               | P2 de <code>effectuer_depot</code> | C2      |
| T8  | Dépôt               | données invalides            | P3 de <code>effectuer_depot</code> | C2      |

|     |         |                    |                         |        |
|-----|---------|--------------------|-------------------------|--------|
| T9  | Dépôt   | dépôt valide       | P4 de effectuer_depot   | C1, C2 |
| T10 | Retrait | compte introuvable | P1 de effectuer_retrait | C2     |
| T11 | Retrait | compte inactif     | P2 de effectuer_retrait | C2     |
| T12 | Retrait | données invalides  | P3 de effectuer_retrait | C2     |
| T13 | Retrait | solde insuffisant  | P4 de effectuer_retrait | C2     |
| T14 | Retrait | retrait valide     | P5 de effectuer_retrait | C1, C2 |

## 5 Table de progression de couverture

La table suivante indique quels tests permettent de satisfaire les critères C1 et C2 sur les fonctions principales.

| Fonction             | Tests minimaux pour C1      | Tests minimaux pour C2      | Observation                                                                                |
|----------------------|-----------------------------|-----------------------------|--------------------------------------------------------------------------------------------|
| Créer un compte      | T1 + T2                     | T1 + T2                     | une entrée valide et une entrée invalide couvrent les deux branches du test de validation. |
| Lister les comptes   | T3                          | T3                          | pas de décision interne, donc le chemin unique suffit.                                     |
| Consulter un compte  | T4 + T5                     | T4 + T5                     | il faut un compte existant et un compte inexistant.                                        |
| Effectuer un dépôt   | T6 + T7 + T8 + T9           | T6 + T7 + T8 + T9           | quatre chemins sont nécessaires pour couvrir toutes les décisions.                         |
| Effectuer un retrait | T10 + T11 + T12 + T13 + T14 | T10 + T11 + T12 + T13 + T14 | cinq chemins sont nécessaires à cause du test supplémentaire sur le solde insuffisant.     |

## 6 Conclusion

L'analyse du code source montre que la partie la plus simple est la liste des comptes, qui ne contient qu'un chemin d'exécution. Les fonctions de dépôt et de retrait sont les plus importantes pour l'étude C1 et C2, car elles comportent plusieurs décisions successives. Pour le retrait, cinq chemins distincts doivent être identifiés afin de couvrir l'absence du compte, l'inactivité du compte, l'invalidité des données, l'insuffisance du solde et le cas nominal.

Ainsi, la suite du devoir peut s'appuyer directement sur ce document pour présenter :

- les fonctionnalités du logiciel ;
- les graphes de flot de contrôle construits à partir du code source ;
- les différents chemins d'exécution ;
- la table de progression de couverture pour C1 et C2.